Qwen

Deconstructing the Blueprint: A Technical Analysis of Block Architecture in Super Mario Construct

The Data-Driven Architecture of Block Behavior

The functionality of blocks in Super Mario Construct (SMC) is not derived from monolithic, hard-coded object classes but rather from a sophisticated, data-driven architecture centered around a series of interconnected JSON files [88]. This modular approach decouples the visual representation, physical properties, and behavioral logic of blocks, allowing for immense flexibility in level design without requiring changes to the core game engine. The foundation of this system lies in the separation of concerns across several key file types, each serving a distinct purpose in defining an object's identity and capabilities. The primary files identified in the provided context—`TileBrushes.json`, `TileCollisions.json`, `TileEdgeCollisions.json`, `WrapCollisions.json`, and `TileLayouts.json`—collectively form a comprehensive blueprint for every interactive element in the game world.

At the base of this hierarchy is the `TileBrushes.json` file. This file serves as the definitive catalog of a block's visual identity, linking a unique tile ID to its corresponding graphical asset [38]. In modern 2D game development, assets are often stored in "sprite sheets" or "tilesets," which are single image files containing numerous smaller images packed together efficiently [124][157]. A sprite sheet might contain all the graphics for platforms, bricks, coins, and various enemy sprites [129]. The `TileBrushes.json` file acts as a map or atlas for this resource, specifying for each tile ID where on the sprite sheet to find its image and potentially defining its dimensions and origin point [168]. For example, a brick block might be assigned a tile ID of 5, which points to a 16x16 pixel section at coordinates (16, 0) on the main tileset image [176]. This system is highly efficient, as it allows the game engine to render vast environments by simply referencing a small integer (the tile ID) instead of managing hundreds of individual image objects [55]. The existence of separate brush and collision files strongly suggests a design philosophy where a single visual asset can be associated with multiple sets of behavioral rules, enabling powerful creative freedom for level designers [72].

Building upon this visual foundation is the suite of collision files, which dictate how entities interact with the physical world. These files represent the most critical layer for understanding block functionality, as they translate a static tile ID into a dynamic set of physical rules. The primary file, `TileCollisions.json`, likely defines the standard, non-specialized collision properties for each tile. It would contain flags or parameters indicating whether a tile is solid, passable, or possesses other basic properties [40]. A solid tile would prevent any entity from passing through it, forming the basis of all level geometry, while a passable tile would allow free movement, often used for backgrounds or decorative elements [6]. The more nuanced behaviors are handled by supplementary files. `TileEdgeCollisions.json` is specifically designed to manage interactions with edges and slopes [73]. Instead of requiring a unique sprite for every possible corner piece, this file allows the engine to detect when an entity is near an edge-defined tile and apply modified physics, such as altered gravity or friction, to simulate ramps and half-pipes [8]. This technique reduces the asset overhead required for creating visually smooth and continuous surfaces, a hallmark of polished platformer design [33]. Finally, `WrapCollisions.json` is dedicated to a specialized class of blocks responsible for level progression, namely warps [59]. This file contains the instructions for teleportation, defining the destination map, coordinates, and player state upon exiting a warp pipe or door [150]. By isolating these different collision scenarios into separate files, the SMC engine maintains a clean and scalable architecture, capable of supporting a wide array of block types—from simple walls to complex interactive objects—using a consistent underlying mechanism.

The final pillar of the architectural model is the rendering and animation logic, governed by the `TileLayouts.json` files, such as `TileLayouts_SMAS1.json` and `TileLayouts_SMW.json`. While `TileBrushes.json` provides the static visual asset, the layout files dictate the dynamic presentation. They contain metadata that instructs the engine on how to animate a given tile [154]. For instance, an animated tile like flowing lava or rippling water would have its sprite sequence defined within this file. The data would specify a list of frames, their respective offsets on the sprite sheet, the duration for which each frame should be displayed, and whether the animation should loop continuously [158][171]. This allows for the creation of rich, living environments without increasing the number of draw calls or the complexity of the core rendering loop. Furthermore, these layout files likely manage the new layer system introduced in SMC V8, defining the Z-index or layer assignment for tiles to determine their visual stacking order [174]. This ensures that foreground elements like pipes appear in front of background scenery like clouds and mountains. The modularity of this system means a designer could, in theory, change a block's entire appearance and animation by simply swapping its associated layout file, demonstrating the profound flexibility afforded by this data-

centric approach. The combination of these three layers—a visual definition, a collision specification, and a rendering/animation directive—creates a powerful, decoupled framework that is the cornerstone of level creation in Super Mario Construct.

File Type	Primary Function	Key Concepts
`TileBrushes.json`	Defines the visual asset (sprite) for a tile ID [38].	Links a tile ID to a position on a sprite sheet; manages static visuals.
`TileCollisions.json`	Defines standard solid/non-solid collision properties for tiles.	Basic passability flags; forms the foundation of level geometry.
`TileEdgeCollisions.json`	Manages collision for edges and slopes.	Enables ramps and angled surfaces without unique corner sprites [73].
`WrapCollisions.json`	Defines teleportation logic for warps (pipes, doors).	Specifies destination maps and coordinates for level progression [59].
`TileLayouts_*.json`	Controls animation and layering for tiles.	Defines frame sequences, timing, looping for animations [154]; manages visual stacking order [174].

Collision Systems: From Solid Geometry to Specialized Interactions

The collision system in Super Mario Construct is a multi-layered framework meticulously defined by the provided extra tile collision files. This system moves far beyond simple, binary passable/solid checks, employing a nuanced set of rules to govern every interaction between the player, enemies, and the block-based environment. Understanding this architecture is paramount for any level designer seeking to create predictable and engaging gameplay. The core of this system resides in how the engine interprets tile IDs through the lens of `TileCollisions.json`, `TileEdgeCollisions.json`, and `WrapCollisions.json`.

At its most fundamental level, collision is determined by a tile's passability. A tile marked as solid will present an impassable barrier to any entity that collides with it, preventing movement through its boundaries [40]. This is the default behavior for most ground and wall tiles, forming the structural skeleton of every level. Conversely, a passable tile allows entities to move through it freely, typically used for background elements that provide visual context without affecting gameplay [6]. However, the true sophistication of the system emerges in its ability to handle directional and contextual collisions. The classic coin block exemplifies this principle perfectly. A coin block is only broken when the player character, Mario, collides with it from above while moving downward [11]. This

logic is not inherent to the block itself but is a rule applied by the engine based on the player's state at the moment of impact. When a collision occurs, the engine checks the player's velocity vector. If Mario's vertical velocity is positive (indicating he is falling), the collision is treated as a head-butt against the top of the block, causing him to bounce slightly and triggering the block's destruction and item-dispensing animation [34]. If, however, Mario is moving upward (e.g., after a jump) and hits the block from below, the collision is registered as an attack, and Mario takes damage [37]. This directional awareness is a critical gameplay mechanic that prevents players from exploiting ceilings to fly indefinitely. This logic is almost certainly encoded within `TileCollisions.json` through flags or states associated with the specific tile ID for the coin block, telling the engine which type of collision response to trigger based on the incoming vector of the colliding object.

The `TileEdgeCollisions.json` file elevates the collision system beyond simple rectangular prisms, enabling the creation of complex geometries like ramps and slopes. This file allows the engine to define special collision rules for the edges of tiles, providing a way to simulate non-tiled corners and angled surfaces [8]. When Mario's hitbox intersects with a tile flagged in this file, the engine applies a modified set of physics calculations. Instead of simply stopping his horizontal or vertical movement, it calculates his trajectory along the surface of the slope, allowing him to walk up or slide down. This creates a much smoother and more natural-feeling platforming experience compared to a system that relies solely on discrete, grid-aligned tiles. The implementation likely involves calculating the angle of the intersecting edge and adjusting Mario's gravity vector relative to that angle, a technique common in advanced 2D physics engines [84]. This capability significantly expands the creative possibilities for level designers, allowing them to construct intricate courses with varied terrain without being constrained by a rigid square grid. The use of Axis-Aligned Bounding Boxes (AABB) for collision detection is a standard practice in games of this nature, where each object is represented by a simple rectangle aligned with the x and y axes, making collision tests computationally inexpensive [5] [86].

Finally, the `WrapCollisions.json` file serves a completely different purpose: it acts as a scripting tool for level progression and transition. Rather than defining general physics, this file establishes the rules for teleportation [59]. Certain tiles, designated as warp entrances (like pipes and doors), are linked to entries in this file. When the player enters one of these tiles, the engine consults `WrapCollisions.json` to find the corresponding exit point. The data specifies the target screen or map to load, the exact X and Y coordinates for the player's spawn, and potentially other state-altering information, such as granting invincibility frames or altering Mario's power-up status upon arrival [150]. This system is the backbone of many level design techniques, including shortcuts, secret

areas, and connecting different parts of a level. Advanced players in similar games have developed complex contraptions using these mechanics, sometimes leading to paradoxical situations where a level becomes mathematically impossible to complete [21] [23]. The existence of this file demonstrates that some blocks are not merely environmental props but are integral components of the game's narrative and structural flow, acting as portals that transcend the immediate geometry of the current screen.

Gameplay Dynamics: Character, Enemy, and Object Interactions

Blocks in Super Mario Construct are not passive scenery; they are active participants in the game's dynamic systems, engaging in a constant dialogue with the player character, enemies, and other game objects. The rules governing these interactions are embedded within the block's data definitions, transforming a simple tile into a functional element of gameplay. These interactions range from fundamental platforming mechanics like stomping enemies to more complex behaviors involving specialized blocks that release items or spawn distant entities.

The most iconic interaction is the stomp kill, where Mario defeats an enemy by landing on its head. This mechanic is a direct result of the collision system's directional awareness. When Mario, who has a downward vertical velocity, collides with an enemy whose bounding box is positioned above Mario's, the engine triggers a "stomp" event [34] [140]. This is distinct from a head-butt, where Mario would take damage if he were moving upward and collided with the enemy from below. The block representing the enemy must possess a specific property flag that allows it to be defeated by this method. Similarly, breakable blocks, such as Brick Blocks or Question Blocks, respond to player attacks. As previously discussed, a jump from above is the standard trigger, but some blocks may have different durability or require multiple hits. The developer changelog note regarding Flip Blocks provides insight into more complex interactions: when a Flip Block is activated, it releases any sprites contained within it [173]. These released sprites are now placed in a 'tossed' state, suggesting a scripted sequence of events tied to the block's action, such as applying an initial velocity to the newly freed object. This indicates that some blocks are not just static entities but are containers for other objects, adding another layer of interactivity to level design.

Enemy interactions with blocks further expand the gameplay possibilities. While enemies primarily interact with the player, they also collide with the level geometry. An enemy running into a solid wall will reverse direction, a behavior governed by the same collision logic that affects Mario [2]. More interestingly, certain enemies can destroy blocks. For example, a Koopa Troopa hitting a Brick Block might cause it to break. This destructive property would be defined as part of the enemy's data profile, perhaps as a "destructive" flag that, when active, allows it to pass through or break specific tile types. The context also mentions electricity interacting with enemies and objects, implying the existence of specialized damaging blocks [182]. An "electrified floor" block, for instance, would have a script attached to its `TileLayout` or `TileBrush` data that applies damage to any entity touching it. This creates timed hazards and adds strategic depth to navigating certain areas of a level. The distinction between different enemy behaviors is crucial; a Goomba might be fragile and die on contact with Mario, while a Hammer Bro. might be larger and able to break blocks, reflecting the diverse AI and physical properties defined for each creature [139].

Perhaps the most advanced interactions occur when blocks act upon other blocks, blurring the line between static scenery and dynamic game objects. The Track Block is a prime example of this meta-mechanic. In games like Super Mario Maker, Track Blocks can be used to load entities from across the map, a technique known as SALTI (Spawning And Loading Things Instantly) [63]. This implies that a Track Block is not merely a platform but a command block. When Mario steps on it, the engine executes a hidden script that spawns an enemy, item, or even another block elsewhere on the map. This mechanic transforms blocks from simple building pieces into tools for creating large-scale, interconnected level systems. Another example is the Warp Box, which functions similarly to a Warp Door by teleporting the player to a matching box, directly utilizing the `WrapCollisions.json` data for its function [150]. These specialized blocks demonstrate that the block system in SMC is highly extensible. A level designer can leverage these interactions to create puzzles, traps, and surprising moments that go far beyond basic platforming. For instance, a puzzle could require the player to use a Track Block to activate a switch located in a distant, inaccessible area, or a trap could involve flipping a block to release a projectile that destroys an enemy on the other side of a room. The ability for blocks to generate, manipulate, and transport other game objects is a testament to the power of SMC's data-driven design, empowering creators to build experiences that are both mechanically deep and creatively boundless.

Visual Rendering and Animation: Creating the Game World

The visual manifestation of blocks in Super Mario Construct is meticulously controlled by a combination of sprite sheets, tileset definitions, and animation data specified in the `TileLayouts.json` files. This rendering pipeline is designed for efficiency and flexibility, allowing developers and level designers to assemble complex and visually engaging worlds from a limited set of graphical assets. The process begins with the foundational concept of tiling, where the game world is constructed from small, reusable graphical units [15] [176]. These units, often 16x16 pixels in size, are collected into a single image file known as a sprite sheet or tileset [157]. This method is highly efficient, as it minimizes the number of texture loading operations the GPU needs to perform, which is critical for maintaining high performance, especially in browser-based games [152]. The `TileBrushes.json` file serves as the essential bridge between the abstract world of tile IDs and the concrete world of pixels, mapping each ID to a specific region on the sprite sheet [70].

Beyond static visuals, the `TileLayouts.json` files are responsible for bringing blocks to life through animation. Many elements in a Mario-style game are not static; lava flows, water ripples, and certain power-ups pulse with energy. This is achieved by defining animation sequences directly within the layout files [154]. For an animated tile, such as a lava block, the JSON file would contain a data structure specifying a series of frames. Each frame is essentially a reference to a different portion of the sprite sheet, akin to a flipbook animation [130]. The file would also define the duration for which each frame is displayed before advancing to the next, controlling the animation's speed, and a boolean value indicating whether the sequence should repeat continuously [171]. The engine's rendering loop then automatically cycles through these frames, creating the illusion of fluid motion. This system allows for the creation of dynamic backgrounds and hazardous zones that make the game world feel more alive and immersive. For example, a large area of water could be rendered using a single animated tile, avoiding the performance cost of creating and managing dozens of individual animated sprite objects [159].

The introduction of advanced features like autotiling, mentioned in the context of Snapshot 34, further enhances the efficiency and quality of level creation [175]. Autotiling is a feature that automates the placement of corner and edge tiles. Instead of manually placing a specific "top-left corner" tile to connect two perpendicular straight walls, the editor can automatically select and place the correct corner piece based on the surrounding tiles [81]. This not only dramatically speeds up the process of building

complex structures but also ensures cleaner, more professional-looking results by eliminating awkward gaps or misaligned edges. This functionality is likely integrated into the tilemap editor's user interface, working behind the scenes to modify the raw tilemap data. The new layer system introduced in SMC V8 complements this by providing explicit control over the visual stacking order of tiles [174]. Designers can assign different tiles to different layers, ensuring that foreground elements like pipes and platforms are always rendered on top of background scenery like clouds and mountains. This separation improves both visual clarity and rendering performance, as the engine can draw background layers first and then overlay foreground elements. Together, these rendering technologies—the use of sprite sheets, data-driven animation, autotiling, and a multi-layered rendering pipeline—provide a robust and flexible toolkit for constructing visually rich and technically sound game worlds in Super Mario Construct.

Level Progression and Architectural Design

In Super Mario Construct, blocks serve a dual purpose: they are both the building blocks of the physical environment and the primary tools for architects to shape the player's journey. Their role extends far beyond mere obstacles or platforms; they are instrumental in guiding progression, controlling pacing, and implementing complex level structures. The game's mechanics for warps, checkpoints, and path obstructions are all realized through specialized blocks whose behavior is defined by the collision and layout files. This makes blocks a fundamental instrument for storytelling and challenge design within the game's world.

The most direct application of blocks in progression is the use of warp mechanics. Using the `WrapCollisions.json` file, a level designer can designate specific tiles—typically pipes and doors—as entry points to a teleportation system [59]. When the player collides with one of these tiles, the engine reads the associated data to determine the destination. This can be used to create a variety of effects, from simple shortcuts that allow players to bypass difficult sections to elaborate hub-and-spoke level designs [103]. The concept of using pipes to travel between screens is a staple of the Super Mario franchise, and SMC's implementation via a dedicated JSON file shows a commitment to this legacy while providing the flexibility to create custom warp destinations [62]. Advanced level designs can even incorporate glitches or troll-level tactics, such as a warp that leads to an earlier checkpoint, forcing the player to replay significant portions of the level [104]. The Warp Box offers a similar functionality, teleporting the player to a matching box elsewhere in the

level, further expanding the toolbox available for creating non-linear and challenging layouts [150].

Checkpoints are another critical block-based mechanic for managing player progression and mitigating frustration. A checkpoint block, typically a small flagpole, saves the player's progress [105]. If the player dies after passing a checkpoint, they will respawn from that point rather than from the beginning of the level or the last previous checkpoint. The implementation of this feature involves a combination of collision detection (triggered when Mario touches the flagpole) and game state management. The provided context hints at even greater complexity, mentioning password-based checkpoints [148]. This suggests that different checkpoint blocks could have unique identifiers or trigger different save states, allowing for intricate level designs where the player might need to choose between multiple paths, each with its own independent checkpoint system. The careful placement of checkpoints is a core tenet of good level design, balancing challenge with fairness by ensuring that a long, difficult section doesn't lead to a single, punishing death spiral [61].

Beyond these explicit progression tools, the fundamental placement of solid blocks dictates the flow of the level. Platforms guide the player forward, creating a clear path to follow. Walls and ceilings obstruct paths, forcing the player to explore alternative routes or backtrack. This basic architectural use of blocks can be enhanced with conditional logic. For example, a path can be blocked off by a Brick Block that can only be destroyed after the player obtains a specific power-up, or by a Warp Door that remains sealed until a nearby switch is activated [153]. This turns blocks into integral components of puzzles, requiring the player to solve a problem before they can proceed. The ability to strategically place blocks to create tension, reward exploration, and control the rhythm of gameplay is what separates a simple collection of platforms from a well-designed, memorable level. Whether used to create a linear, fast-paced run or a sprawling, interconnected maze, blocks remain the foundational material of level architecture in Super Mario Construct.

Mechanic	Associated Block(s)	Purpose in Progression	Key Files / System
Warp Pipes/ Doors	Pipe, Warp Door	Teleports the player to a different location in the same or a different screen.	`WrapCollisions.json` [59], `TileLayouts`
Checkpoints	Flagpole	Saves the player's progress, allowing them to respawn from this point after dying.	Game State Management [105], Changelog notes on password-based CPs [148]
Path Obstruction	Brick Block, Wall	Creates barriers that can be permanent or temporary, guiding or restricting player movement.	`TileCollisions.json`, Conditional Logic
Secret Areas	Hidden Blocks, Warp Doors	Conceals alternate routes, collectibles, or shortcuts, rewarding exploration.	Tile Placement, Warp Mechanics [62]

Synthesis and Strategic Application for Level Creation

The comprehensive analysis of Super Mario Construct's game files reveals that its block system is a masterclass in data-driven game design. The functionality of every block—from a static platform to a dynamic, interactive object—is not hardcoded but is instead defined by a set of modular JSON files. This architecture, which decouples visual assets (`TileBrushes.json`), collision rules (`TileCollisions.json`, `TileEdgeCollisions.json`), and rendering logic (`TileLayouts.json`), provides an exceptionally flexible and powerful framework for level creation. The collision system, in particular, is a multi-layered construct that handles everything from simple AABB-based solid geometry to sophisticated edge-based slopes and specialized warp scripts, enabling a rich tapestry of physical interactions [5] [73]. This foundation supports a dynamic ecosystem where blocks interact meaningfully with characters, enemies, and even other blocks, giving rise to emergent gameplay possibilities like item release, enemy spawning, and large-scale environmental manipulation [63] [173].

For a level creator, this technical understanding translates directly into actionable strategies for designing more compelling and technically proficient levels. First and foremost, mastery of the collision files is essential. Before placing a single tile, a designer should study `TileCollisions.json`, `TileEdgeCollisions.json`, and `WrapCollisions.json` to understand the properties and behaviors associated with each tile ID. This foreknowledge allows for precise placement and predictable outcomes, preventing frustrating bugs where a block behaves unexpectedly. Second, designers should move beyond viewing blocks as mere scenery and embrace their potential as interactive tools. Specialized blocks like Flip Blocks, Track Blocks, and electrified floors are not just props; they are verbs in the language of level design, capable of releasing

items, triggering distant events, and creating timed hazards [63] [173] [182]. Leveraging these mechanics allows for the creation of puzzles and challenges that are deeper and more engaging than simple platforming gauntlets.

Furthermore, the new layer system introduced in V8 provides an opportunity to enhance both the visual polish and performance of levels [174]. By thoughtfully assigning tiles to different layers—background, midground, and foreground—a designer can create a sense of depth and improve rendering efficiency. This attention to visual hierarchy helps guide the player's eye and makes complex level layouts easier to parse. Finally, blocks should be wielded as instruments of architectural design to control the player's journey. The strategic use of warps and checkpoints can transform a level from a linear obstacle course into a more open and exploratory experience [59] [105]. Warps can create shortcuts and secret areas, while well-placed checkpoints can mitigate the sting of failure in particularly difficult sections, encouraging players to persevere. By animating background tiles, designers can also imbue the world with a sense of life and atmosphere, making the environment itself a more engaging presence [154]. In essence, the provided files reveal that Super Mario Construct is not just a game to be played, but a toolkit to be mastered. Its block system, built upon a foundation of transparent and accessible data, empowers creators to architect dynamic, interactive, and memorable experiences.

Reference

1. Mario game implementation based on SDL2 - GitHub https://github.com/Phoenix-flame/Mario/
2. super-mario-bros/index.html at main - GitHub https://github.com/The-User-Python/super-mario-bros/blob/main/index.html
3. n64decomp/sm64: A Super Mario 64 decompilation ... - GitHub https://github.com/n64decomp/sm64
4. reruns/mario: A javascript clone of Super Mario Bros. for the NES https://github.com/reruns/mario
5. kikito/bump.lua: A collision detection library for Lua - GitHub https://github.com/kikito/bump.lua
6. GitHub - bjorn/super-mario-war https://github.com/bjorn/super-mario-war

7. NSMBW-Community/Puzzle-Updated: The Puzzle Tileset Editor ... https://github.com/NSMBW-Community/Puzzle-Updated
8. GitHub - HackerN64/HackerSM64: A Super Mario 64 decompilation ... https://github.com/HackerN64/HackerSM64
9. alinr19/Super-Mario-Game - GitHub https://github.com/alinr19/Super-Mario-Game
10. HerbFargus/Super-Mario-War: A simplified build of Super ... - GitHub https://github.com/HerbFargus/Super-Mario-War
11. Super Mario Brothers Tile collision checks flowchart : r/programming https://www.reddit.com/r/programming/comments/rdvy23/super_mario_brothers_tile_collision_checks/
12. SMC v8.beta29.4 - Changelog - Super Mario Construct https://levelsharesquare.itch.io/super-mario-construct/devlog/878221/smc-v8beta294-changelog
13. The Mario Maker 3 Wishlist: Quality of Life and game-wide changes https://www.reddit.com/r/MarioMaker/comments/1emj3j3/the_mario_maker_3_wishlist_quality_of_life_and/
14. Useful info for people making traditional SMB1-style levels. - Reddit https://www.reddit.com/r/MarioMaker/comments/3qg7yw/on_technical_limitations_useful_info_for_people/
15. ELI5: How did game developers for old games like Super Mario put ... https://www.reddit.com/r/explainlikeimfive/comments/6o3ji5/eli5_how_did_game_developers_for_old_games_like/
16. Super Mario Bros - "fixed" powerups : r/nes - Reddit https://www.reddit.com/r/nes/comments/6gig5e/super_mario_bros_fixed_powerups/
17. What are some things you want in a hypothetical 3rd Mario Maker ... https://www.reddit.com/r/Mario/comments/19eg8rc/what_are_some_things_you_want_in_a_hypothetical/
18. Mysteries of Mario Maker 2 : r/MarioMaker - Reddit https://www.reddit.com/r/MarioMaker/comments/cshqy9/mysteries_of_mario_maker_2/
19. You know what item we really need?: Fuzzies. : r/MarioMaker - Reddit https://www.reddit.com/r/MarioMaker/comments/dbspnq/you_know_what_item_we_really_need_fuzzies/
20. Super Mario Bros. Wonder Direct 8.31.2023 : r/nintendo - Reddit https://www.reddit.com/r/nintendo/comments/166bzt2/super_mario_bros_wonder_direct_8312023/
21. How a Group of Hackers Broke into the Most Secure Mario ... https://www.youtube.com/watch?v=u9127T9yasc

22. Mario Maker Hack: Vertical Levels : r/MarioMaker https://www.reddit.com/r/MarioMaker/comments/8dv6ly/mario_maker_hack_vertical_levels/
23. Is it Possible to Upload a Mathematically Impossible Level in ... https://www.youtube.com/watch?v=aSlstPpIW-E
24. Super Mario Construct - Snapshot 31 Devlog [June 29 2025] https://www.youtube.com/watch?v=fvIRv1vlqsI
25. Super Mario Maker's Slow Unlocking Feels More ... http://www.nintendoworldreport.com/editorial/40974/super-mario-makers-slow-unlocking-feels-more-frustrating-than-fun
26. How to Make A “Backwards Only” Level in Super Mario Maker 2! https://www.youtube.com/watch?v=BlgGgo56zqw
27. Super Mario Construct/settings https://wiki.levelsharesquare.com/Super_Mario_Construct/settings
28. How the Mario Maker Community Solved “Don't Look Back” ... https://www.youtube.com/watch?v=h-KaXWpHCs8
29. Reverse Design: Super Mario World https://thegamedesignforum.com/features/RD_SMW_1.html
30. Creating a Super Mario game engine and level editor https://www.facebook.com/groups/692686004114836/permalink/4964065593643501/
31. Super Mario Bros Programming Tutorial - CS50's Intro to Game ... https://www.youtube.com/watch?v=_cPwFo--1LA
32. I'm having some difficulty with the code for collision with ... https://forum.gamemaker.io/index.php?threads/im-having-some-difficulty-with-the-code-for-collision-with-moving-platforms.15672/
33. Handling collisions in Super Mario Brothers with bounding boxes https://www.youtube.com/watch?v=dhoPTY-ogbk
34. Mario style: Jump on top of enemy to kill it https://www.construct.net/en/forum/construct-2/how-do-i-18/mario-style-jump-top-enemy-40391
35. Super Mario Bros. - CS50's Introduction to 2D Game ... https://cs50.harvard.edu/games/projects/4/mario/
36. Super mario bros "rounded" tile collision detection https://gamedev.stackexchange.com/questions/182348/super-mario-bros-rounded-tile-collision-detection
37. Trying to make a player bounce of an enemy like Mario - Help https://forum.godotengine.org/t/trying-to-make-a-player-bounce-of-an-enemy-like-mario/121001

38. Super Mario Construct developer tools https://github.com/Level-Share-Square/SMC-tools
39. Trying to replicate Super Mario Maker with C++ https://github.com/paolodelia99/mario-maker
40. Lecture 4: Super Mario Bros. - CS50's Introduction to Game ... https://cs50.harvard.edu/games/2018/notes/4/
41. SMC-released-sprites https://github.com/Level-Share-Square/SMC-released-sprites/
42. How do I Destroy tile jumping, Super Mario style? https://www.construct.net/en/forum/construct-2/how-do-i-18/destroy-tile-jumping-super-80876
43. Combat Mechanics | Slay the Spire Wiki - Fandom https://slay-the-spire.fandom.com/wiki/Combat_Mechanics
44. What is one of the most busted mechanics you've seen in a ... https://www.reddit.com/r/TwoBestFriendsPlay/comments/1hufdt7/what_is_one_of_the_most_busted_mechanics_youve/
45. SF Game Design Notes: Dodge, Parry, and Jump ... https://www.linkedin.com/pulse/sf-game-design-notes-dodge-parry-jump-mechanics-rpgs-scott-fine-elmgc
46. Adding an Attack Mechanic for Stomping in Unity Playground https://www.youtube.com/watch?v=rD8V7fCSjZY
47. Simple changes that reduce reasons for toxic interactions https://en-forum.guildwars2.com/topic/163310-simple-changes-that-reduce-reasons-for-toxic-interactions/
48. Best way to stop players from endlessly wailing on an enemy. https://gamedev.net/forums/topic/705767-best-way-to-stop-players-from-endlessly-wailing-on-an-enemy/
49. Combat Mechanics - Sea of Stars Guide https://www.ign.com/wikis/sea-of-stars/Combat_Mechanics
50. MTFS - Mechanics https://www.dustloop.com/w/MTFS/Mechanics
51. Stunning Detail: Hitstun in Depth - Celia Wagar's CritPoints https://critpoints.net/2016/08/14/stunning-detail/
52. Sambo3975/SMBX2-Tileset-Creator - GitHub https://github.com/Sambo3975/SMBX2-Tileset-Creator
53. Releases · JHDev2006/Super-Mario-Bros.-Remastered-Public https://github.com/JHDev2006/Super-Mario-Bros.-Remastered-Public/releases
54. bobzhang/MariOCaml - GitHub https://github.com/bobzhang/MariOCaml
55. GitHub - JuniorTour/es6-mario: A web game made by ES6 syntax ... https://github.com/JuniorTour/es6-mario

56. GitHub - colinvella/phaser-tilemap-plus https://github.com/colinvella/phaser-tilemap-plus
57. NcJump: a jumping project · nCine · Discussion #10 - GitHub https://github.com/nCine/nCine/discussions/10
58. Miyamoto/puzzle.py at master · aboood40091/Miyamoto · GitHub https://github.com/aboood40091/Miyamoto/blob/master/puzzle.py
59. Super Mario Construct V8: Warp & Checkpoint Tutorial https://www.youtube.com/watch?v=9muN0SYZDGY
60. How to Customize Moving Platforms - Super Mario Construct ... https://www.youtube.com/watch?v=MvznFUrTo9U
61. What are your level design "rules"? : r/MarioMaker https://www.reddit.com/r/MarioMaker/comments/3ks7eq/what_are_your_level_design_rules/
62. Warp Door | Super Mario Maker 2 Wiki https://super-mario-maker-2-wiki.fandom.com/wiki/Warp_Door
63. How to Unleash the Loading Magic of the Track Block in Mario ... https://www.youtube.com/watch?v=FaGJxqtvfdQ
64. Super Mario Maker: Do's and Don'ts - General Gaming https://forums.serenesforest.net/topic/57998-super-mario-maker-dos-and-donts/
65. Code Super Mario in JS (Ep 5) - Tile Collision https://www.youtube.com/watch?v=YLMP5jmtpYc
66. A Comprehensive Super Mario Bros. Disassembly - GitHub Gist https://gist.github.com/1wErt3r/4048722
67. GitHub - rose-mtz/Super-Mario-Bros https://github.com/Rosendo-Martinez/Super-Mario-Bros
68. pulkitcs21/MARIO-Game: Making World 1-1 of Mario Using Java and ... https://github.com/pulkitcs21/MARIO-Game
69. Remaking Super Mario World in Godot (Part 2) https://www.youtube.com/watch?v=Zo3Me6KwlRI
70. Modular Game Worlds in Phaser 3 (Tilemaps #1) https://medium.com/@michaelwesthadley/modular-game-worlds-in-phaser-3-tilemaps-1-958fc7e6bbd6
71. Lecture 4 - CS50's Introduction to 2D Game Development https://cs50.harvard.edu/games/notes/4/
72. How do I correctly make collisions for tiles? https://www.construct.net/en/forum/construct-2/how-do-i-18/correctly-collisions-tiles-106868
73. Dev Log, Week 2: Collision, Tiles, and Not-So-Simple Geometry https://u.osu.edu/solidlight/2017/05/20/dev-log-week-2/

74. FranzNL/rusty-mario: A Rust port of the timeless python ... - GitHub https://github.com/FranzNL/rusty-mario/
75. Checkpoint Glitch in Super Mario Maker https://www.youtube.com/watch?v=BRwG4YMnakk
76. Any way to effectively revisit the same checkpoint twice in ... https://www.reddit.com/r/MarioMaker/comments/ccbimt/any_way_to_effectively_revisit_the_same/
77. Super Mario Construct V8 Snapshot 16 Overview https://www.youtube.com/watch?v=eyHxNqJP5yc
78. Why this Two Pixel Gap is Among the Most Complicated Things ... https://www.reddit.com/r/Games/comments/b28fn5/why_this_two_pixel_gap_is_among_the_most/
79. How to pass 2nd level on Super Mario with blocked tunnel ... https://www.facebook.com/groups/312585526091341/posts/1337066643643219/
80. Mario 3 wrong warp glitch. Every step must be done exactly as ... https://www.instagram.com/reel/DZWrfpUS0rz/
81. Tile map editor for game development with object ... https://www.facebook.com/groups/SpectrumNextDev/posts/2773805349664133/
82. Tilemap collisions? - Game Building Help https://www.construct.net/en/forum/construct-2/how-do-i-18/tilemap-collisions-135447
83. How To Make A Game #6 : Implement Tilemap's : C++ And ... https://www.youtube.com/watch?v=1eaxE_waDNc
84. 14. C++ AABB Collision and Movement - Celeste Clone https://www.youtube.com/watch?v=IaUcAt0jDqs
85. Optimizing AABB continuous collision detection with a ... https://community.monogame.net/t/optimizing-aabb-continuous-collision-detection-with-a-large-tilemap/13455
86. How To Make A Game #11 : Collision Detection - AABB in C++ ... https://www.youtube.com/watch?v=rjirHFVQAkY
87. GitHub - Level-Share-Square/SMC-translations https://github.com/Level-Share-Square/SMC-translations
88. GitHub - manfredsteyer/mario https://github.com/manfredsteyer/mario
89. sm64/src/engine/surface_collision.c at master - GitHub https://github.com/n64decomp/sm64/blob/master/src/engine/surface_collision.c
90. GitHub - Developers-Collective/Puzzle-Next: A tileset, tile animation ... https://github.com/Developers-Collective/Puzzle-Next

91. GitHub - TabularisDB/game: A free Super Mario-style browser ... https://github.com/TabularisDB/game
92. Super Mario Hear Attack is a big example on how to use ... - GitHub https://github.com/JonathanPicques/Super-Mario-Heart-Attack
93. Special Monthly Compensation (SMC) "Cheat Sheet" - Reddit https://www.reddit.com/r/VeteransBenefits/comments/1jp0d13/special_monthly_compensation_smc_cheat_sheet/
94. M21-1, Part VIII, Subpart iv, Chapter 4, Section A - Special Monthly ... https://www.knowva.ebenefits.va.gov/system/templates/selfservice/va_ssnew/help/customer/locale/en-US/portal/554400000001018/content/554400000177480/M21-1,-Part-VIII,-Subpart-iv,-Chapter-4,-Section-A---Special-Monthly-Compensation-(SMC)
95. Discover SMC Benefits — Apply Today https://www.veteransbenefitskb.com/smc
96. VA Special Monthly Compensation (SMC) - MyArmyBenefits https://myarmybenefits.us.army.mil/Benefit-Library/Federal-Benefits/VA-Special-Monthly-Compensation-(SMC)?serv=128
97. What Are the Easiest SMC-K Claims? | Hill & Ponton, P.A. https://www.hillandponton.com/additional-compensation-smc-k/
98. Special Monthly Compensation (SMC) https://www.howvadisabilityratingswork.com/resources/special-monthly-compensation-smc
99. VA Special Monthly Compensation (SMC) Level R - CCK Law https://cck-law.com/blog/va-special-monthly-compensation-smc-level-r/
100. What is VA Special Monthly Compensation (SMC)? https://www.veteransdisabilityinfo.com/blog/what-is-va-special-monthly-compensation-smc/
101. A Special Monthly Compensation: Who Qualifies & 2026 Rates https://vetclaims.ai/va-benefits/disability-pay/special-monthly-compensation/
102. Special Monthly Compensation: SMC(k) Explained - CCK Law https://cck-law.com/blog/special-monthly-compensation-series-smck/
103. Super Mario Maker - Checkpoint! : r/Games https://www.reddit.com/r/Games/comments/3qf6pq/super_mario_maker_checkpoint/
104. Trolling Techniques - Super Mario Maker 2 Wiki - Fandom https://supermariomaker2.fandom.com/wiki/Trolling_Techniques
105. Super Mario Maker and the Devious Joy of the Troll Level https://medium.com/@KevinHainline/super-mario-maker-and-the-devious-joy-of-the-troll-level-fdb1894aa7e1
106. A Very Mean Mario Maker Checkpoint https://kotaku.com/a-very-mean-mario-maker-checkpoint-1762010738

107. Code Super Mario in JS (Ep 11) - JSON Entity Spawning - YouTube https://www.youtube.com/watch?v=y99x_sBEeP8
108. Super Mario Runner with Vanilla JavaScript - GitHub https://github.com/romantech/super-mario
109. citylex-2026-06-15_morphology_segmention.tsv - GitHub https://github.com/connoryang331/morphoneme/blob/main/data/citylex-2026-06-15_morphology_segmention.tsv
110. passman/js/vendor/zxcvbn/zxcvbn.js.map at master - GitHub https://github.com/nextcloud/passman/blob/master/js/vendor/zxcvbn/zxcvbn.js.map
111. sdcg/corpora/brown/tagwords.pl at master · cth/sdcg · GitHub https://github.com/cth/sdcg/blob/master/corpora/brown/tagwords.pl
112. cs-178-project/imdb.vocab at main · apmalani/cs-178-project https://github.com/apmalani/cs-178-project/blob/main/imdb.vocab
113. unsupervised_topic_modeling/topics/en/15/100/50/topics at ... https://github.com/annontopicmodel/unsupervised_topic_modeling/blob/master/topics/en/15/100/50/topics
114. python-zxcvbn/zxcvbn/generated/frequency_lists.json at master ... https://github.com/dropbox/python-zxcvbn/blob/master/zxcvbn/generated/frequency_lists.json
115. maltego/top100Kenglishwords.txt at master · michenriksen ... https://github.com/michenriksen/maltego/blob/master/top100Kenglishwords.txt
116. How to make Super Mario Bros in Unity (Part 1) - Level Design https://www.youtube.com/watch?v=GCkq6XqyJZg
117. Super Mario War https://github.com/mmatyas/supermariowar
118. PolyMarsDev/Mario-Maker-Deconstructed https://github.com/PolyMarsDev/Mario-Maker-Deconstructed
119. Remaking COINS and COIN BLOCKS in Unity - Mario Maker ... https://www.youtube.com/watch?v=qovHBHuj5QA
120. You Can't Solve These Super Mario Bros. Levels ... https://arxiv.org/html/2405.10546v1
121. Construct 2 Tutorial - Super Mario Bros: The Levels (Part 13) https://www.youtube.com/watch?v=TYFN2Wq0KYg
122. I figured out how graphics work today. I stole the Mario trick ... https://www.facebook.com/groups/NESmakers/posts/1388882238132764/
123. SMBX64 - Moondust Wiki https://wohlsoft.ru/wiki/index.php?title=SMBX64&mobileaction=toggle_view_desktop

124. sprite sheets and data abstraction [SDL3 Episode 19] https://www.youtube.com/watch?v=innm0MCTlas
125. Sprite sheet animation editor that has more than basic gif ... https://www.reddit.com/r/gamedev/comments/3i0y3f/sprite_sheet_animation_editor_that_has_more_than/
126. How should I go about handling nonuniformed sprite sheet ... https://gamedev.stackexchange.com/questions/168634/how-should-i-go-about-handling-nonuniformed-sprite-sheet-with-json-data
127. Sprite Sheet Basics Explained https://spritesheetgenerator.online/guides/sprite-sheet-basics
128. Spritesheets https://pixijs.com/7.x/guides/components/sprite-sheets
129. Creative Coding 1, MART 120 | Sprites https://montana-media-arts.github.io/creative-coding-1/modules/week-13/sprites/
130. 2D Animation - Cornell: Computer Science https://www.cs.cornell.edu/courses/cs5152/2025sp/lectures/lecture7/slides-7.pdf
131. Phaser Multi-Row Sprite Sheets — Frames & Atlases ... https://www.youtube.com/watch?v=m_UToxF1vTY&vl=en-US
132. How to create sprite sheets and animations with MonoGame https://www.codeandweb.com/texturepacker/tutorials/how-to-create-sprite-sheets-and-animations-with-monogame
133. Pixi.js Sprite Animation https://medium.com/@simonhfrost/pixi-js-sprite-animation-1546867f64ae
134. Super Mario Construct Tutorial - All Customizable Enemies (Water ... https://www.youtube.com/watch?v=-qfvYgkVZEY
135. This Enemy Spawns BOSSES [Super Mario Construct] - YouTube https://www.youtube.com/watch?v=JHKv1iQ1viQ
136. Super Mario Construct v8 Snapshot 34: Developer Stream - YouTube https://www.youtube.com/watch?v=qozl0AfxhQQ
137. Super Mario Construct V8 Snapshot 33 Preview: Rollin' Enemies! https://www.youtube.com/watch?v=rLO2vDlUNVE
138. Super Mario Construct v8 - Snapshot 29 Developer Stream - YouTube https://www.youtube.com/watch?v=jXBURkmwmq0
139. How make iconic Mario enemies in construct 3 ... - YouTube https://www.youtube.com/watch?v=MvRLfyYouwc
140. How to make a Enemy Elimination System/Mario Stomp ... - YouTube https://www.youtube.com/watch?v=sdtz3JxMXzU
141. Super Mario Construct - CUSTOMIZABLE HALLOWEEN ... - YouTube https://www.youtube.com/watch?v=hug5IXu0X90

142. What is Super Mario Construct? | How to Make & Share Levels https://www.youtube.com/watch?v=dEliEk5noSo
143. Super Mario Construct V8: Halloween Snapshot - YouTube https://www.youtube.com/watch?v=WkdKYHKglDM
144. Super Mario Construct V8 Snapshot 32 Preview https://www.youtube.com/watch?v=Ex72TL5ZR5A
145. Collision detection in "compressed" levels https://www.lexaloffle.com/bbs/?tid=2368
146. mario-game · GitHub Topics https://github.com/topics/mario-game?l=c%2B%2B&o=asc&s=stars
147. mario-game · GitHub Topics https://github.com/topics/mario-game
148. How to add multiple password-based Checkpoints to your ... https://www.reddit.com/r/MarioMaker/comments/3mrshx/how_to_add_multiple_passwordbased_checkpoints_to/
149. SMM2: Tech, Glitches, and Contraptions | PDF | Mario https://www.scribd.com/document/422644201/Common-Contraptions-Setups-Tech-And-Glitches-in-SMM2
150. Warp Box | Super Mario Maker 2 Wiki https://super-mario-maker-2-wiki.fandom.com/wiki/Warp_Box
151. Super Mario Maker: How to Unlock Everything & Tips https://www.spoilermedia.net/2016/01/23/super-mario-maker-unlock-everything/
152. Question: Is there a way to use this with `@pixi/spritesheet`? https://github.com/pixijs-userland/tilemap/issues/163
153. Placing props on a map - Questions https://discourse.mapeditor.org/t/placing-props-on-a-map/6399
154. Animated Tile | 2D Tilemap Extras | 4.2.1 https://docs.unity3d.com/Packages/com.unity.2d.tilemap.extras@4.2/manual/AnimatedTile.html
155. Chapter 13: Working with Tilemaps https://docs.monogame.net/articles/tutorials/building_2d_games/13_working_with_tilemaps/index.html
156. Creating Animated Tiles for your Tilemap in Unity - Tutorial https://www.youtube.com/watch?v=65IrtBEZeVs
157. Day 18: Sprites & Tilemaps – Understanding Game Engines https://courses.ideate.cmu.edu/53-353/m2023/index.html%3Fp=206.html
158. Implementing animated tiles from Tiled JSON Tilemap files https://www.html5gamedevs.com/topic/33542-implementing-animated-tiles-from-tiled-json-tilemap-files/
159. How do i create tilemap with animation https://forum.gdevelop.io/t/how-do-i-create-tilemap-with-animation/73883

160. How to Make TileSets with Animated Sprites for Use in TileMaps https://www.reddit.com/r/godot/comments/sab2yy/how_to_make_tilesets_with_animated_sprites_for/
161. Level Share Square - GitHub https://github.com/Level-Share-Square
162. ericfrederich/smlinux: Super Mario Linux - FAQ and Builder Installer ... https://github.com/ericfrederich/smlinux
163. Adds touch support for Super Mario Construct! · GitHub https://github.com/WINRARisyou/SMC-TouchControls
164. Writing an Unsupervised Learning Super Mario Bros 1 AI - Part 1 #1 https://github.com/d12/blog/issues/1
165. GitHub - WINRARisyou/SMC-Desktop-Mod-Manager: Way to ... https://github.com/WINRARisyou/SMC-Desktop-Mod-Manager
166. Super Mario Recreation https://ece4760.github.io/Projects/Spring2025/mtw82_njw66/index.html
167. Construct 2 Tutorial - Super Mario Bros: Item Blocks (Part 7) https://www.youtube.com/watch?v=wEsBm1KIlUY
168. Sprite sheet support - #9 by Kerempooh - Feature requests https://forum.gdevelop.io/t/sprite-sheet-support/71808/9
169. How To Make A Mario Animation | Tutorial https://www.youtube.com/watch?v=xJDWx2-El8A
170. Working on a <100kb game right now, and honestly ... https://www.facebook.com/groups/132728896890594/posts/3423048447858606/
171. JSON Format · JHDev2006/Super-Mario-Bros. https://github.com/JHDev2006/Super-Mario-Bros.-Remastered-Public/wiki/JSON-Format/ae403515f44a552a8b5cfb9676d4ce35fe30ad3e
172. Sprite - Construct 3 Documentation https://www.construct.net/en/make-games/manuals/construct-3/plugin-reference/sprite
173. SMC v8.beta30 - Changelog - Super Mario Construct https://levelsharesquare.itch.io/super-mario-construct/devlog/957849/smc-v8beta30-changelog
174. Overview of The New Layer System - Super Mario Construct (V8) https://www.youtube.com/watch?v=SzfoYKII9fU
175. SMC v8.beta34 - Changelog - Super Mario Construct https://levelsharesquare.itch.io/super-mario-construct/devlog/1197230/smc-v8beta34-changelog
176. Super Mario Fusion: Revival Level Design Tutorial https://mkfusion.fandom.com/wiki/Super_Mario_Fusion:_Revival_Level_Design_Tutorial
177. Super Mario Construct V8: NEW SMB1 GAME STYLE! #1 - YouTube https://www.youtube.com/watch?v=lkCB2gl3G-E

178. Planning Mario Maker Levels on your computer with Tiled Map Editor https://www.reddit.com/r/MarioMaker/comments/3mzn6d/planning_mario_maker_levels_on_your_computer_with/
179. Level Share Square https://levelsharesquare.com/games/supermarioconstruct
180. Super Mario Construct - NEW Customizable Enemies ... - YouTube https://www.youtube.com/watch?v=1ecSMRWwUEo
181. Super Mario Construct V8 Snapshot 32 Preview: Bullies, and...? https://www.youtube.com/watch?v=c0s45fL7KUk
182. Super Mario Construct (PC) - Electricity Vs. All Enemies [SMW][v.8 ... https://www.youtube.com/watch?v=MDNaboEDQTs
183. Java to Bedrock 3D Model Converter 1.1 - GitHub Gist https://gist.github.com/Kastle/1ceffdcef37aaaeffe938d97ec522ea5
184. awesome-swift-2/README.md at master - GitHub https://github.com/zenangst/awesome-swift-2/blob/master/README.md
185. GitHub - bradtraversy/design-resources-for-developers: Curated list ... https://github.com/bradtraversy/design-resources-for-developers
186. GitHub - teetteet/collected_stars: A list of all projects that rock my world https://github.com/teetteet/collected_stars
187. starred/by-language.md at master · b0o/starred - GitHub https://github.com/b0o/starred/blob/master/by-language.md
188. GitHub - jerrypm/awesome-swift-2 https://github.com/jerrypm/awesome-swift-2
189. design-resources.md - GitHubのGist https://gist.github.com/bgoonz/d0165281fa3c7974d745aecae57fe517
190. dennysjmarquez/awesome-css - GitHub https://github.com/dennysjmarquez/awesome-css
191. Human readable descriptions for file extensions. Scraped from https ... https://gist.github.com/alexgleason/8338add8a0a3366577c2
192. stylus-2018-09-25.json - GitHubのGist https://gist.github.com/watain666/481e0b0713c336f0ab663d3c12103d2d